

Transformations and Trajectories of a Content-Addressed State Machine

The Emerging World Model HLLSet Architecture

(TECS submission draft)

Alex Mylnikov

Alexandr Mylnikov Jr.

Deependra Kumar

October 8, 2026

Abstract

Contemporary AI systems are stateless at inference time: each request is answered from a context window that is rebuilt from scratch, and “memory” is a buffer rather than a state. We describe **ewm-state-machine**, an engineering realization of *Emerging World Models* (EWM) in which the state itself is a content-addressed set. The state is an **HLLSet**—a compact bit-vector sketch of a token collection over a fixed 32K-bit plane—and every transformation of the machine is an operation on a Boolean lattice: union joins, intersection meets, symmetric difference is ring addition. The paper’s center of gravity is the **transformation** and the **trajectory**: a single turn ingests a token stream, updates the working state $S(t)$ by a join, pushes the incoming original into a GF(2) ring whose basis is *monotone* (every commit HLLSet stays in the base’s span), commits the result to a content-addressed store, and materializes tokens back for the host. We separate the temporal roles cleanly— $S(t)$ is the emerging state, $H(t-1)$ the committed base, and $D/R/N$ the departed/retained/new decomposition—and we organize the running record as a set of *frames* whose coordinates form the state trajectory. We present the architecture (foundation crates for HLLSets and morphisms; state crates for the store, cache, processing unit, explorer, and side-car hosts), the transformation algebra (extension and rotation over a moving basis, exact change of basis between commits), and a trajectory toolkit with labeled exactness: canonical coordinates, novelty, covers, and minimal covers. We then give the view that makes the trajectory readable: the ring basis is a *rotating coordinate system*, and each commit’s top state is decomposed in the coordinates current when it arrives—a soft key (the projection onto each base coordinate), a hard key where the state lies in the span, and a residual for the off-span part. The same property dictates what the store keeps: only *originals* and *base HLLSets* are materialized, while every compound—union, $D/R/N$, residual, or projection—is recorded as a provenance-stamped *view* (an expression plus the SHA-1 identifiers of its sources) and recreated against the base current at read time. On a 10×10 synthetic clip the linear residual flags all nine scene cuts, the bit-novelty signal separates cuts from in-scene drift by $3.75\times$, symmetric differences are always representable in the ring, and intersections/unions/differences are the measurable obstruction. The rotating-coordinate trajectory reads out the transition structure directly: the basis frontier grows monotonically, 534 coordinates re-pivot over 100 commits, and the pre-commit residual spikes at every scene cut. The result is a reproducible, vocabulary-agnostic state layer that any model can use as a side-car.

Keywords: content-addressed state; HLLSet; Boolean lattice; GF(2) linear algebra; state machine; trajectory; side-car memory; HyperLogLog.

1 Introduction

A language model, a vision model, or a planner is a function: it maps an input to an output and keeps nothing. What we call its “memory” is a text buffer or a key–value cache that is

re-derived for every request. This is convenient, but it means the system has no *state* in the control-theoretic sense: there is no value that is advanced, committed, and recovered; no notion of what was retained, what departed, and what is new. Rebuilding context from scratch is also where much of the cost and most of the failure modes of long-horizon systems live.

This paper describes **ewm-state-machine**, a state layer built on one unusual choice: **the state is a content-addressed set**, not a tensor. A token stream is measured into an **HLLSet** [3, 1]: a fixed-length bit vector whose set bits are the sites the tokens hashed into (32,768 bits = 4 KB at the default geometry). Because subsets of a common plane form a Boolean lattice [4], every state and every derived quantity—unions, intersections, differences, novelty, residuals—is a node of one lattice, addressed by content (IICA: idempotent, immutable, content-addressed). Nothing is created that was not already a node; a computation *discovers* a node by naming a lattice expression [1].

The consequence is a state machine whose transformations are cheap and exact set operations, and whose trajectory is a sequence of nodes that can be replayed, compared, and audited. The processing unit is stateless and disposable: it reads a tip, ingests a turn, proposes the next state, and forgets. The state lives *outside* it, in a shareable cache and a persistent store. Recovery is a “pop, not a rebuild”: discard the unit and its cache, restore from the tip, and continue; uncommitted work is reprocessed, and idempotence makes that safe.

Contributions.

1. **A state-machine formulation.** We give the fundamental equation $H(t) = (S(t), H(t-1), D, R, N)$ and the four ideas that make it safe (one universe; the morphism is the contract; the fiber is global; decompositions are frames)—§3.
2. **An architecture.** We describe the layered realization: foundation crates for HLLSets, contracts, LUTs, and morphisms; state crates for the content-addressed stack, the cache, the processing unit, the explorer, the operational graph, and the side-car hosts—§4.
3. **Transformations.** We specify one turn exactly: encode, ingest, join, ring-push, commit, materialize. The ring basis is *monotone* (every commit HLLSet stays in the base’s span), so the span only grows and representability is non-decreasing; a change of basis transports coordinates exactly across commits—§5.
4. **Views, not stored compounds.** The same transport property fixes the storage discipline: only originals and base HLLSets are materialized, while every compound is a provenance-stamped view recreated against the base current at read time—§5.5.
5. **Trajectories and tools.** We formalize readings (frames) and their coordinates as the state trajectory, and we present a toolkit with explicit exactness: canonical coordinates, novelty, bit-novelty via covers, and minimal covers for context compression—§6.
6. **Rotating coordinates and the top-state decomposition.** We show that the ring basis is a rotating coordinate system and decompose each commit’s top state in the coordinates current when it arrives—soft key, hard key, and residual—which makes the transition structure readable as a coordinate–time matrix (base coordinates on Y , commits on X)—§7.
7. **Reproducible evidence.** On a 10×10 scene clip the residual detects all nine cuts, the bit-novelty signal separates cuts from in-scene drift by $3.75\times$, and the representability laws are measured (symmetric difference 394/394 in span; intersection/union/difference 0/394)—§8.

The paper is deliberately engineering-first. The mathematics is kept to the minimum needed to say what the machine does and why it is safe; the full lattice/world theory is in the companion note [1] and the main EWM paper [2].

2 HLLSets and the Boolean lattice

2.1 What an HLLSet is

An **HLLSet** is a compact sketch of a token collection. Instead of storing tokens, it stores a fixed-length bit vector whose set bits are the *measurement sites* the tokens hashed into. The geometry is *soldered* [3, 1]: precision P fixes the register count $M = 2^P$; each register occupies b bits; a token t is hashed once and decomposed into a register index and a trailing-zero state,

$$\text{reg}(t) = h(t) \bmod M, \quad \text{tz}(t) = \text{clamp}(\text{trailing_zeros}(h(t) \gg P), 0, b - 1), \quad (1)$$

naming a single **bit address**

$$\text{bit}(t) = \text{reg}(t) \cdot b + \text{tz}(t) \in \{0, \dots, M \cdot b - 1\}. \quad (2)$$

With the defaults $P = 10$, $b = 32$, the plane has $M \cdot b = 32,768$ bits (4 KB packed). The sketch of a token collection $A \subseteq \mathcal{T}$ is

$$H(A) = \bigcup_{t \in A} \{\text{bit}(t)\}, \quad (3)$$

so a collection is represented by the sites it marks, not by the tokens themselves. The bit vector deliberately forgets provenance: it does not record which token or which scheme set a bit.

2.2 One lattice

The fact that turns sketches into a representation is that subsets of a common plane are automatically a lattice.

Definition 2.1 (HLLSet lattice). *Let $\mathbb{B} = \{0, \dots, M \cdot b - 1\}$ be the bit plane. The HLLSet lattice is the Boolean lattice*

$$\mathcal{B} = (2^{\mathbb{B}}, \subseteq, \cup, \cap, \Delta),$$

with order $\subseteq$ (bitwise subset), join $\cup$ (bitwise OR), meet $\cap$ (bitwise AND), and ring addition Δ (symmetric difference).

Every computation in the machine—unions, the retained set $R = A \cap B$, the $D/R/N$ decomposition, ring residuals, basis frames—names a node of $\mathcal{B}$ and *discovers* it rather than creating it. Two paths that name the same expression reach the same node; that is the operational meaning of “one universe”.

2.3 The morphism is the contract (IICA)

The token-to-node connection is a *chosen* measurement morphism $\mu : \{\text{token collections}\} \rightarrow \{\text{HLLSets}\}$. The model requires only that μ be **IICA**: Idempotent (the same collection always yields the same bits), Immutable (bits are never mutated in place; only new nodes appear), and Content-Addressable (a node is named by a digest of its content). Any algorithm satisfying IICA is interchangeable. Two useful instances appear in this paper:

- the **flat projection** μ_{flat} , which is a join homomorphism: each token sets its own bits, so $\mu_{\text{flat}}(A \cup B) = \mu_{\text{flat}}(A) \cup \mu_{\text{flat}}(B)$;
- **windowed n -gram ingestion** μ_{ng} , used by the state machine, which is only inclusion-isotonic: $A \subseteq B \Rightarrow \mu_{ng}(A) \subseteq \mu_{ng}(B)$. Union identities survive as inclusions, not equalities.

Both are valid IICA morphisms; they are different *measurements*, and the lattice facts below hold in whichever image the chosen morphism produces.

Remark 2.2 (Discovered, not created). A lattice node is a name for a set expression. The phrase “the state is content-addressed” is therefore literal: the state is the digest of the bits, and two computations that reach the same bits reach the same state.

3 The EWM concept: state, transformation, trajectory

3.1 The fundamental equation

The state machine is organized around one equation, in which every term is an HLLSet:

$$\boxed{H(t) = (S(t), H(t-1), D, R, N)} \quad (4)$$

with

$S(t)$ = current emerging state (work in progress), $H(t-1)$ = previous committed state (the base),

$N = S(t) \setminus H(t-1)$ (new), $R = S(t) \cap H(t-1)$ (retained), $D = H(t-1) \setminus S(t)$ (departed).

The decomposition is a partition of the union of the two states into three disjoint pieces: N , R , and D . It is the temporal signature of a transition: what the new turn brings, what it keeps, and what it drops.

3.2 Four ideas, in order of depth

The model hangs on four ideas. They are the minimal context a reader (or an engineer) needs before touching the code.

1. **One universe.** Every HLLSet is a node of the same Boolean lattice $\mathcal{B}$ over the bit plane. Nothing new is created; a computation names a lattice expression and discovers the node.
2. **The morphism is the contract (IICA).** The token-to-bit connection is a deterministic, immutable, content-addressed map; only IICA is required, and any algorithm that satisfies it is interchangeable.
3. **The fiber is global.** The same bit in any HLLSet points to the same LUT fiber: materialization is a property of the bit address, never of the HLLSet that contains it. Materialization is exact on the builders and probabilistic on the extras.
4. **Decompositions are frames (readings).** $D/R/N$, joined perceptrons, the ring basis—each is a named set of dimensions through which a node is read. A frame φ_D maps a node to coordinates, and $\varphi_D(X(t))$ over time is the *state trajectory*.

3.3 A state in a stateless system

The apparent contradiction—a state machine whose processing unit keeps no state—is resolved by IICA. $S(t)$ is an immutable, content-addressed value, so it is safe to share and safe to rebuild. The processing unit ([UM], the *processing unit* of the framework) is stateless and disposable; it reads the tip, processes incoming tokens, and proposes the next state. $S(t)$ and $H(t-1)$ live outside it, in a shareable cache backed by a persistent store. Recovery is a *pop*: drop the unit and its cache, restore from the tip, and continue with a fresh unit. Uncommitted work is reprocessed, and idempotence makes reprocessing harmless.

Principle 3.1 (Safe replay). *Because ingestion is idempotent and the join is union, reprocessing an uncommitted turn after a crash yields the same node as processing it once. The committed stack is never mutated; only uncommitted work is redone.*

4 Architecture

The workspace separates a *foundation* layer (HLLSets and morphisms) from a *state* layer (the machine around them). Figure 1 shows the two layers and the data path; Table 1 maps each crate to its responsibility.

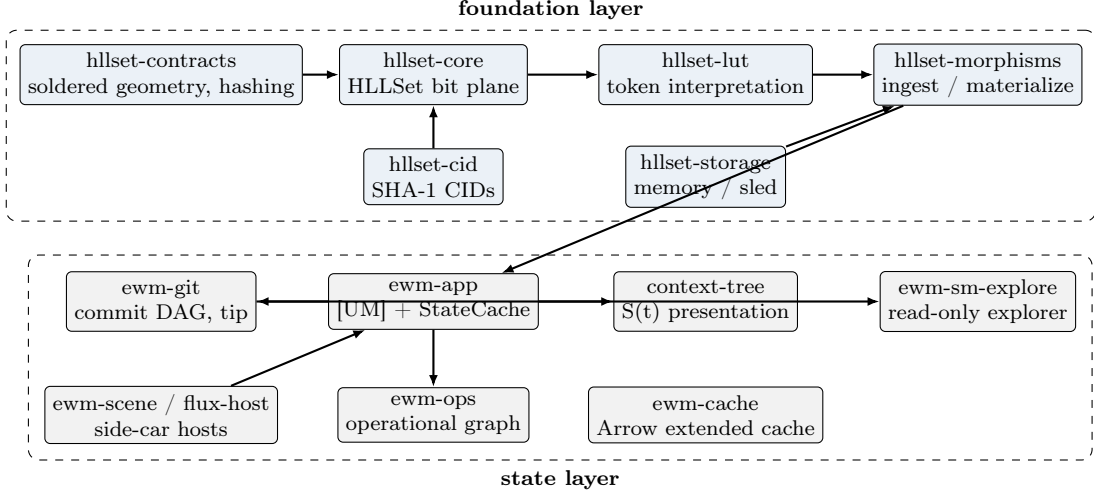


Figure 1: The two layers. The foundation layer knows only HLLSets and morphisms (no notion of a “turn” or a “world”); the state layer builds the machine on top.

Crate	Responsibility
hllset-contracts	soldered invariants: $P = 10$, $M = 1024$, $b = 32$, MurmurHash3, bit addresses
hllset-core	the HLLSet bit plane (a Roaring bitmap) and its set algebra
hllset-lut	the LUT lattice: token interpretation, content-addressed
hllset-morphisms	ingest / materialize and the default API
hllset-cid , hllset-storage	CIDs; memory and sled persistence
ewm-git	the committed stack: commit DAG, tip, recovery
ewm-app	the stateless [UM] harness and the shared StateCache
context-tree	Merkle tree over the working set, the $S(t)$ presentation
ewm-scene	LLM $\leftrightarrow$ HLLSet side-car helper (direct morphisms)
ewm-flux-host	side-car host adapter for a rectified-flow sampler
ewm-ops	content-addressed operational graph (value / program lattice)
ewm-cache	Arrow-backed extended cache (IPC spill/restore)
ewm-sm-explore	read-only explorer of the three-layer state

Table 1: Implementation map. The foundation layer is vocabulary-agnostic; the state layer is where the machine lives.

4.1 The side-car loop

A host model (an LLM, an OCR encoder, a vision model, a planner) is left untouched. The state layer attaches beside it:



The encoder is the only vocabulary-aware step: it quantizes host encodings to a shared codebook and emits $\text{tid}\{n\}$ tokens. Everything below it is vocabulary-agnostic: any stream that can be tokenized can drive the same state machine. This is what makes the layer a *side-car* rather than a model: it does not interpret content, it accumulates and reads structure.

4.2 Store, cache, unit

The state machine is one data structure over three locations:

- the **persistent store** (`ewm-git`)—the committed stack, a content-addressed DAG whose head is the tip;
- the **shared cache** (`StateCache`)—the emerging $S(t)$ working set together with the materialized base $H(t-1)$;
- the **processing unit** (`ewm-app`)—stateless and disposable, owning only a store handle, reading the tip, proposing the next state.

Caching and persistence are separated by content addressing: the cache is a pure value, shareable between units, and rebuildable from the tip. A commit is the atomic advance of the tip and records $H(t)$; a no-change pass commits nothing. What the store actually persists is deliberately narrow—*originals* and *base HLLSets*—with every compound held as a provenance-stamped view and recomputed on read; §5.5 derives that discipline from the change of basis.

5 Transformations

5.1 One turn

A turn is the unit of transformation. Given a token list, the machine performs the following steps; each step is a lattice operation, a Gaussian step, or a commit.

1. **Encode.** Host encodings are quantized to a `tid` stream by the shared codebook encoder (the only vocabulary-aware step).
2. **Ingest.** The stream is measured by the IICA morphism; the turn produces per-channel sketches. The current projection is the turn’s *original* H_T .
3. **Join.** The working state advances by union: $S(t) \leftarrow S(t) \cup H_T$. Union is idempotent, commutative, and associative, so independent turns accumulate without coordination.
4. **Ring push.** The original enters the ring (§5.2); the push is one Gaussian step and returns novelty statistics.
5. **Commit.** $H(t)$ is committed when the pass brings new bits *or* the ring basis changed; a basis change alone is a structural commit.
6. **Materialize.** On demand, bits are lowered back to tokens (LUT-first, probabilistic restoration) for the host.

The $D/R/N$ decomposition is computed from $S(t)$ and $H(t-1)$ as in Eq. (4). It is the per-transition reading that later frames build on.

5.2 The ring transformation

The ring is the part of the machine that turns the sequence of originals into a *coordinate system*. We call the HLLSet a commit pushes its *commit HLLSet*, and an element of the ring basis a *base HLLSet*. An HLLSet is a subset of the bit plane and hence a vector over $GF(2)$: symmetric difference is addition, $x + x = 0$, and intersection is multiplication, $x \cdot x = x$. The ring maintains a *basis* of the span of the originals—a row-echelon $GF(2)$ basis with distinct leading bits—and each original is reduced against it.

A push is one Gaussian step. Reducing the incoming original against the basis either (i) returns zero, meaning the original is already an XOR of basis elements (*in span*), or (ii) leaves a non-zero *residual*, which becomes a new basis element with a new pivot. The basis change then has two components, which the machine reports as `RingStats`:

- **extension**—the residual becomes a new direction (the span grows by one);
- **rotation**—every existing basis element containing the new pivot bit is re-pivoted by XOR with the residual, keeping leading bits distinct.

The number of re-pivoted elements is `rotation_count`; the total Hamming change of the old basis is `rotation_mass = rotation_count × residual`. An in-span push changes nothing.

5.3 A monotone basis and exact change of basis

The ring keeps two roles separate: a **monotone basis** over every commit HLLSet ever pushed, and a bounded **visibility window** of recent commits used as context. Eviction moves the window; it never removes a base HLLSet. Consequently the span is *monotone*,

$$\langle B_1, \dots, B_k(t-1) \rangle \subseteq \langle B_1, \dots, B_k(t) \rangle, \quad (5)$$

and representability is non-decreasing: once a set is expressible as an XOR of the basis, it remains so. Every commit HLLSet remains expressible in the base.

Because the span only grows, each old basis element has unique coordinates in the new basis: the columns of a **change-of-basis matrix** M . A coordinate vector a at time $t-1$ transports to $a \cdot M$ at time t , exactly. This is what keeps coordinates canonical across commits: the coordinates of a fixed node change with the basis, but the change is a deterministic, invertible (in the pure re-basis case) linear map, and the ring stamps each basis with a *generation* so stale projections can be identified.

Proposition 5.1 (Exact transport). *Let $B(t-1)$ and $B(t)$ be two bases of a monotone ring. Every basis element of $B(t-1)$ has unique coordinates in $B(t)$, so any node expressible at $t-1$ is expressible at t , and its coordinates transport by the change-of-basis matrix.*

Proof sketch. Monotonicity gives $\langle B(t-1) \rangle \subseteq \langle B(t) \rangle$; coordinates are unique because the basis is distinct for each commit; the transport is the coordinate expansion of each affected old basis element in the new basis. $\square$

5.4 What the ring cannot do

The ring span is a vector subspace over $GF(2)$: it is closed under symmetric difference but *not* under intersection, union, or difference. For A, B in the span,

$$A \cup B = A \Delta B \Delta (A \cap B), \quad A \setminus B = A \Delta (A \cap B),$$

so all three compounds lie in the same coset modulo the span and are representable together or not at all. The meet is the single obstruction; a general lattice expression is representable iff its meet content is. The exact test for any node X is $\text{res}(X) = \emptyset$, equivalently $\text{coordinates}(X) \neq \text{None}$. This is a feature, not a defect: it gives a measurable quantity—how far a node is from being a linear combination of the context.

Remark 5.2 (Remainder, not canonical representative). The reduction’s remainder is deterministic for a fixed commit sequence but is only *a* representative of the coset $X + \langle B \rangle$: the basis is row-echelon without full reduction, and the reduction stops at the first non-pivot leading bit. Congruent sets can return different remainders. The coset-invariant fact—and the one the tests rely on—is $\text{res}(X) = \emptyset \Leftrightarrow X$ is in the span.

5.5 Views: originals, base HLLSets, and recreated compounds

The transport property is not only coordinate bookkeeping; it is the reason the store keeps so little. Two kinds of node are *materialized*:

- **originals**—the IICA sketches of measured token streams, content-addressed ($\mathbf{h}:\langle \text{SHA-1} \rangle$) and immutable;
- **base HLLSets**—the monotone basis the ring produces, themselves content-addressed.

Everything else—unions $S(t)$, the $D/R/N$ components, residuals, soft keys, covers, and projections—is a **view**: a node recorded as an *expression* over source nodes together with its **provenance**, the ordered list of SHA-1 content identifiers of the HLLSets it is built from. A view is not stored as a copy of its bits.

Node	Stored as	Read as
original sketch	bits, $h: \langle \text{SHA-1} \rangle$	itself
base HLLSet	bits, $h: \langle \text{SHA-1} \rangle$	itself
union, $D/R/N$	expression + provenance CIDs	recomputed against the read-time base
residual, soft key, cover, pro- jection	expression + provenance + basis generation	recomputed; cached entries are generation-stamped

Table 2: What is stored and what is recomputed. Only originals and base HLLSets are materialized; every compound is a provenance-stamped view that is recreated against the base current at read time.

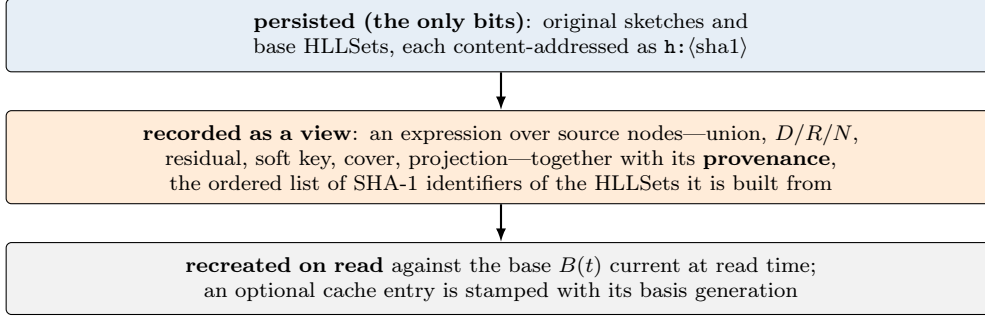


Figure 2: The storage discipline. Only originals and base HLLSets are materialized; a compound is a view—a recipe plus provenance—and it is recreated against the read-time base. Because the basis rotates, a stored coordinate or bit copy would be stale; the recipe never is.

The reason is exactly that coordinates are basis-relative and the basis moves (§7): a compound frozen as a coordinate vector in one base, is different in another. The correct rule is the transport of Proposition 5.1—a reading is valid only relative to the *generation* of the basis it was taken in. So the store keeps the recipe and its provenance, and the reader *recreates* the compound against the basis current at read time. Cache is the only place where compound HLLSets stay binary vectors.

Recreation is safe because the source nodes are IICA: the same recipe on the same sources yields the same bits, so a view is deterministic and idempotent under re-evaluation. It is also cheap: a view over k base HLLSets costs a handful of bit operations per base HLLSet, independent of how long the history has grown. Caching is optional and subordinate—a materialized view may be cached, but the cache entry is stamped with its basis generation, and a read in a new generation re-projects only the entries queried, never the whole history (the lazy back-propagated pattern used by the flow host). Storage is therefore proportional to the number of *originals* plus the size of the *base*, not to the number of compounds ever named—the operational form of “discover, do not create.”

A view record therefore carries no bits of its own; an abbreviated example:

```

{
  "view":          "p:9f2c1d...",      # id of the recipe, not the result
  "expr":          "union",            # or drn | residual | project | ...
  "sources":       ["h:1a7e...", "h:4b02...", "h:cd91..."],
  "basis_generation": 37,              # generation the reading was taken in
  "materialized":  false               # recomputed on read
}

```

Tool	Exactness
change of basis $B(t-1) \rightarrow B(t)$	exact GF(2) matrix; coordinates transport by $a \cdot M$
BSS transform $BSS(t-1) \rightarrow BSS(t)$	<i>not</i> linear (XOR vs. $\cap$); exact by re-projection, or by the meet-order ladder (order $\geq$ support size); new directions must be measured, not rotated
cover identity projection $X \mapsto (D, R, N)$ onto the cover	exact: $\bigcup \text{basis} = \bigcup \text{originals}$ exact lattice operations; $N \subseteq \text{res}(X)$
minimal cover	unique coordinate support when in span; greedy $H(d)$ -approximate union cover otherwise

Table 3: Trajectory tools and their exactness. The one place to be careful is the BSS transform: a BSS vector is not a sufficient state to rotate exactly; the exact alternative is re-projection, and the nonlinearity is the meet obstruction of §5.3.

6 Trajectory

The trajectory is the sequence of readings of the state. A *frame* is a named set of dimensions $D = (D_1, \dots, D_m)$; the frame’s coordinate map is

$$\varphi_D(X) = (\text{BSS}(X, D_1), \dots, \text{BSS}(X, D_m)), \quad \text{BSS}(X, D_i) = \frac{|X \cap D_i|}{|D_i|}, \quad (6)$$

and $\varphi_D(X(t))$ over t is the HLLSet trajectory in that frame. Different frames are different readings of the same nodes: $D/R/N$ is the temporal frame, the ring basis is the linear frame, joined perceptrons are the structural frame, and a codebook gate is a per-user frame. Varying the frame changes what we see, not what the state is.

6.1 Signals

Four families of trajectory signals are used in the experiments and the host loops.

- **Linear novelty (residual).** $\text{res}(X)$ is the part of X outside the ring span: what no XOR of originals can express. Its popcount is a higher-order N —it asks “what cannot be assembled from the context,” where the bit-level $N = S(t) \setminus H(t-1)$ asks “which bits are new.”
- **Rotation.** `rotation_count` counts how many existing directions re-index themselves through a new one; it is zero whenever the residual is zero, and among novel frames it measures the re-indexing cost.
- **Bit novelty (cover).** The *cover* of the basis is the union of its elements; the identity $\bigcup_i B_i = \bigcup_j (\text{originals})_j$ holds exactly, because an XOR is a subset of a union and every base HLLSet is an XOR of the commit HLLSets. The bit novelty $N_{\text{bit}} = X \setminus \text{cover}$ is the part no span member can contain; it satisfies $N_{\text{bit}} \subseteq \text{res}(X)$.
- **Basis frames (time travel).** Every frame is also projected into the first recorded basis, with a *spill* $|X \setminus \text{cover}(F_0)|$ measuring the error of reading the present through the past’s lens.

6.2 A toolkit with labeled exactness

The companion note and the notebooks develop a small toolkit over the ring and the lattice. Its exactness is part of its specification (Table 3).

6.3 Reading the trajectory

Figure 3 sketches the trajectory picture: nodes advance by join and commit; the ring basis grows; each turn contributes a novelty reading. The machine exposes the trajectory in three

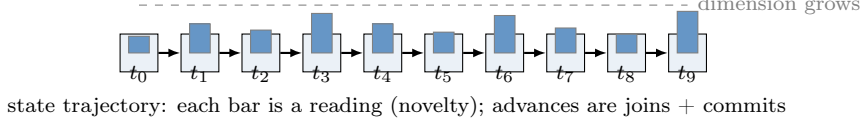


Figure 3: The state trajectory. Nodes advance by join; the ring basis grows monotonically; each turn contributes a novelty reading. Spikes mark transitions (the synthetic clip’s scene cuts in §8).

locational views (run-time $S(t)$, cache, committed store) and in any named frame.

7 Rotating coordinates and the top-state decomposition

The ring’s basis is not a fixed frame. Each commit appends a commit HLLSet (extension) and re-pivots some existing ones (rotation), so the basis $B(t) = \{B_1(t), \dots, B_{k(t)}(t)\}$ *moves*. A node that does not change still changes its coordinates; the transport of Proposition 5.1 is exactly the bookkeeping for that motion. Over the 10×10 clip the basis re-pivots 534 coordinates in 100 commits (a mean of 5.34 per commit), so the frame is moving on almost every step.

7.1 Decomposing the top state

Let $U(t)$ be the *top state* of commit t : the union of the commit’s perceptron sketches (in the single-stream clip, simply the frame sketch $X(t)$). We read it in the coordinates that were current when it arrived, $B(t-1)$ —the context it is measured against—and obtain three exact objects:

- the **soft key**, the lattice projection onto each base coordinate,

$$w_i(t) = \frac{|U(t) \cap B_i(t-1)|}{|B_i(t-1)|} \in [0, 1],$$

always defined, whether or not $U(t)$ is in the span;

- the **hard key**, the unique coordinate vector $c(t)$ with $U(t) = \Delta_i c_i B_i(t-1)$, defined exactly when $\text{res}_{B(t-1)}(U(t)) = \emptyset$;
- the **residual** $\text{res}_{B(t-1)}(U(t))$, the off-span part, which completes the exact decomposition

$$U(t) = P(t) \Delta \text{res}_{B(t-1)}(U(t)), \quad P(t) = \Delta_{i: c_i=1} B_i(t-1).$$

The soft key says *where* the new state lands in the current coordinates; the hard key says whether it lies entirely inside them; the residual says how much of it cannot be expressed by them at all. On the clip no top state is fully inside the prior span (0/100): every frame carries new drift, so the hard key is usually absent and the residual is the informative object. The mean soft-key entry is 0.091 with a maximum of 0.804—most coordinates are barely touched, a few carry the frame—and the pre-commit residual averages 612.6 bits at the nine scene cuts against 328.3 bits in scenes ($1.9\times$).

7.2 The coordinate–time matrix

Arranging the soft keys into a matrix $W \in \mathbb{R}^{T \times k}$, with base coordinates on the Y axis and commits on the X axis, makes the trajectory readable in one picture (Figure 4). Three structures are visible at once:

- the **frontier**—the white line of the span dimension—grows monotonically and bounds the region where no coordinate exists yet;

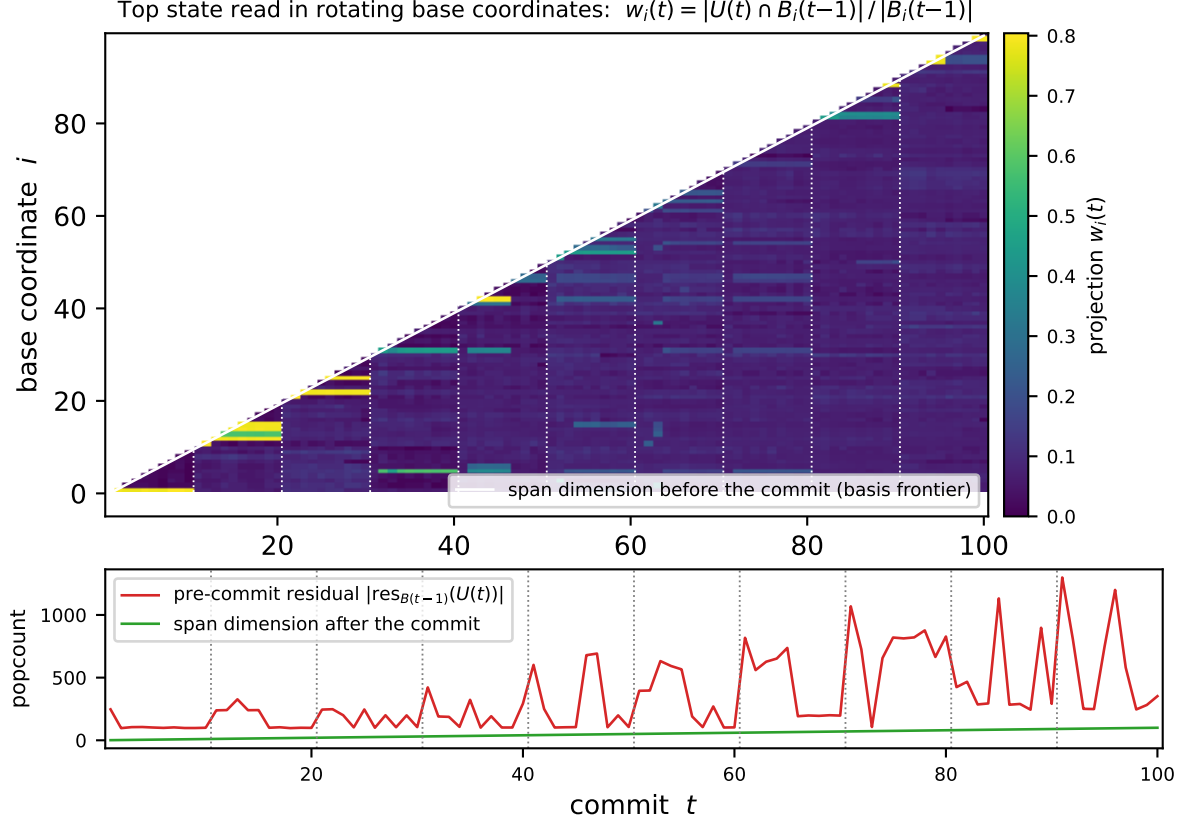


Figure 4: The trajectory in rotating base coordinates on the 10×10 clip. Top: the soft key $w_i(t)$ of each commit’s top state in the coordinates $B(t-1)$ current when it arrived (rows are base coordinates in creation order; the white line is the span frontier; dotted lines are scene cuts). Bottom: the pre-commit residual and the span dimension. Rows are born as the basis grows, the frame re-pivots throughout, and the residual spikes at every scene cut.

- **rows** are born as coordinates appear and then evolve: a bright row is a direction that stays aligned with many commits (a stable context direction); a row that dims has been rotated away from current content;
- the **residual** panel spikes at transitions, giving a representation-independent event signal.

Remark 7.1 (The cumulative union projects to 1). It is tempting to project the *cumulative* union of all HLLSets onto the base coordinates. That projection is uninformative: every base HLLSet is an XOR of commit HLLSets, hence a subset of the cumulative union, so $|\bigcup_{\tau \leq t} U(\tau) \cap B_i(t)| / |B_i(t)| = 1$ for every coordinate—we measure exactly 1.000 for all of them on the clip. The informative object is the *per-commit* top state (or its novel part), which is what Figure 4 shows.

7.3 Reading it in practice

The rotating-coordinate view turns the toolkit of Table 3 into trajectory readings. Coordinates with consistently high soft keys are the stable directions a context can be compressed to (the minimal cover); coordinates that spike and then fade mark transitions; and the residual is the novelty that no combination of current coordinates can express. Because the frame moves, a reading is comparable to another reading only once it is stamped with its basis generation—which is exactly what the ring records.

Signal (morphism, mode)	cut mean	in-scene mean	separation
linear residual, windowed (μ_{ng}) [1]	782.9	271.4	$2.9\times$
linear residual, monotone (μ_{flat})	612.6	328.3	$1.9\times$
bit novelty N , monotone (μ_{flat})	99.0	26.4	$3.75\times$

Table 4: Novelty at scene cuts vs. in-scene drift. The windowed linear residual is the sharpest linear signal on the production n -gram morphism (all 9/9 cuts above in-scene mean $+2\sigma$); the bit-novelty $N = X \setminus \text{cover}$ is the sharpest signal on the flat-morphism clip. The two morphisms are different measurements and their scales are not comparable; the structural conclusions are morphism-agnostic.

Measured on the 10×10 clip	Result
$A\Delta B$ in span	394/394
$A \cap B$, $A \cup B$, $A \setminus B$ in span	0/394
$\cap, \cup, \setminus$ pairwise congruent mod span	394/394
change-of-basis density ($50 \rightarrow 100$)	0.038
old basis elements representable after extension	50/50
coordinate transport round-trip	identity
cover identity $\bigcup \text{basis} = \bigcup \text{originals}$	holds (monotone and windowed)

Table 5: Representability and transport, measured. The ring is a lossy linear projection of the lattice; the residual is the exact error of that projection.

8 Experimental illustration

We exercise the machine on a synthetic clip with known structure: 10 scenes $\times$ 10 frames, where each frame holds a 200-token scene vocabulary plus 56 tokens of per-frame drift. Scene cuts start at frames 11, 21, $\dots$, 91; a window of frames shares vocabulary, and a cut introduces a new one. The clip is analyzed incrementally as frames arrive. The measurements below are reproduced by the notebooks in the repository (a bit-exact Python mirror of the ring on real HLLSets), and by the Rust implementation’s own test suite.

8.1 Novelty detects the transitions

The linear residual is a higher-order N : it spikes when a frame cannot be assembled from anything the context has seen. The bit novelty N is the exact lattice part no span member can contain. Both flag the transitions; N is the more robust reading on the synthetic clip because it is bounded by the cover and does not accumulate re-indexable bits.

8.2 Representability is measurable

The ring’s representability laws are not asymptotic statements; they are measured exactly on the clip (Table 5). Over 394 frame pairs, symmetric differences are *always* in the span, while intersections, unions, and differences are *never* in it for this data; the three compounds are pairwise congruent modulo the span in every case. The meet is the obstruction, exactly as §5.3 predicts. The change of basis between two commits is sparse (density 0.038), all 50 old basis elements are representable in the later 100-dimensional basis, and the coordinate round-trip is the identity—the practical content of Proposition 5.1.

8.3 Trajectory tools

The toolkit of Table 3 is exercised end-to-end. The change of basis transports coordinates exactly; the BSS transform is shown to be nonlinear (a two-element overlap example predicts

1.00 where the exact re-projection is 0.00), and exact by the meet-order ladder when the order reaches the support size; the cover identity holds in both monotone and windowed modes; the projection (D, R, N) reassembles the frame and satisfies $N \subseteq \text{res}(X)$; and the minimal cover compresses a five-frame context of 442 bits into 12 basis elements with full coverage, with the residual as the irreducible remainder.

Remark 8.1 (What the numbers say about the architecture). The machine’s central quantity—the residual—is not a heuristic score bolted onto a buffer; it is the distance from a node to the span of the context, and it is computable exactly, incrementally, and cheaply (a handful of bit operations per basis element). The trajectory is therefore auditable: every reading can be recomputed from the committed originals.

9 Related work

Sketches and cardinality. HLLSets descend from probabilistic cardinality estimation [5, 6] and resemblance sketches [7]. Our use is different: we do not estimate cardinality; we use the sketch as a *state* and its lattice as the algebra of transitions. The soldered geometry fixes a common plane so that set operations are exact bit operations.

Content-addressed storage and histories. Content addressing is the basis of Merkle trees [9], Git [10], and IPFS/IPLD [11]. The state machine reuses the idea at the level of *state*: the tip of a commit DAG is the state, and recovery is a read, not a replay. Merging independent contributions is a join, which relates the approach to CRDTs [12]; here the join is set union on a shared bit plane, and idempotence comes from IICA rather than from a merge function.

World models and memory for models. World models learn a predictive latent state [13]; JEPA-style architectures separate encoder, context, and predictor [14]; retrieval-augmented systems attach a memory to a model [16]. Our side-car is orthogonal: it does not learn the world, it *holds* the state as a lattice node and advances it by transformation. The predictor layer is pluggable and lives above the state layer, not inside it.

10 Discussion and limitations

No canonical basis. $\text{GF}(2)$ has no canonical basis; the ring’s basis depends on ingestion order and pivot choice. Coordinates therefore mean something inside one commit sequence. A canonical alternative—the atoms of the Boolean algebra generated by the originals—is exponential in the number of commit HLLSets and is not used. The monotone basis makes coordinates comparable across a run but not across different ingestion orders.

The remainder is not canonical. The residual is a coset representative, not a canonical one; congruent sets may leave different remainders. The invariant is the coset, and the zero test is the exact representability criterion. Trajectory signals built from the residual should be read as representatives, not as unique values.

Monotone growth has a cost. Keeping every commit HLLSet makes representability monotone and removes eviction churn, but the span dimension grows with the history, and coordinates grow with it. The windowed mode bounds the visibility (and hence the per-frame readings) at the cost of losing representability of evicted originals. The two modes are a deliberate trade; the experiments report both.

Union identities survive only as inclusions. The production n -gram morphism is inclusion-isotonic but not a join homomorphism, so some union identities hold as inclusions rather than equalities. The structural results— Δ closure, the meet obstruction, the cover identity—hold in the image of whichever IICA morphism is chosen.

Recomputation is the price of correctness. Keeping only originals and base HLLSets makes stale-coordinate bugs structurally impossible, but every read of a compound pays its expression cost (a handful of bit operations per base HLLSet). The mitigation is generation-stamped caching—not storing compounds—and the open question is the cache policy that keeps the read cost flat as the base grows.

11 Conclusion and future work

We presented a state machine whose state is a content-addressed HLLSet and whose transformations are lattice operations. The design separates the emerging state $S(t)$, the committed base $H(t-1)$, and the per-transition decomposition $D/R/N$; a stateless processing unit proposes transitions, a shareable cache holds the working state, and a content-addressed store makes the trajectory durable and replayable. The ring transformation gives the trajectory a coordinate system that is deterministic per commit sequence and monotone across commits, together with novelty signals that detect transitions exactly and cheaply.

Several directions follow directly. First, a **trajectory-stability study**: the coordinate-time matrix of §7 can be analyzed as a time series (row birth, drift, and death) to identify stable context directions automatically and to predict transitions from early-warning signals in the soft keys. Second, a **multi-world** extension: the hash configuration is a parameter of the measurement, so independent users can share one lattice while reading it through different interpretations; the companion note develops the isolation and bridge theory. Third, a **closed loop**: the trajectory readings already drive a controller in our host experiments (memory plus curiosity), and the residual and bit-novelty signals are natural reward and surprise channels. Fourth, a **scaled evaluation** on real, long-horizon model streams with the same protocol used here. The engineering claim we make is deliberately modest and testable: any token-producing model can be given a state that is cheap to advance, exact to compare, and safe to replay.

Acknowledgment

This article was written in collaboration between the authors and an AI assistant, following the practice of the companion note. The human authors developed the EWM framework, the HLLSet algebra, and the **ewm-state-machine** realization, and directed the line of argument; the AI assistant helped to draft and structure the text, work through the formal statements, and verify the correspondence with the implementation. The conceptual content is the authors’ own.

A Implementation map

References

- [1] A. Mylnikov, A. Mylnikov Jr., and D. Kumar, *The EWM Lattice and Its Worlds: HLLSets, Emerging Lattices, and Multi-World Representation*, companion note to *Emerging World Models*, 2026.
- [2] A. Mylnikov and D. Kumar, *Emerging World Models: A Structural Framework for Self-Organizing World Representations*, consolidation document, August 2026.

Concept	Artifact
bit plane, HLLSet	hllset-core::HLLSet (Roaring bitmap)
geometry P, M, b	hllset-contracts (P, M, BITS_PER_REG)
bit address	hllset-contracts::BitAddress
morphism μ	hllset-morphisms::ingest / materialize
turn	ewm-app::run_turn
working state $S(t)$	ewm-app::StateCache::working
base $H(t-1)$	commit tip (ewm-git)
$D/R/N$	ewm-scene::noether and the commit decomposition
ring basis, push	ewm-boolring::BoolWindow
ring stats	RingStats{residual, in_span, dimension, rotation_count, rotation_mass}
commit DAG, tip	ewm-git (commit, head)
$S(t)$ presentation	context-tree (Merkle tree over turn leaves)
side-car hosts	ewm-scene (LLM), ewm-flux-host (flow)
operational graph	ewm-ops (value / program lattice)
originals, base HLLSets	content-addressed HLLSet nodes ($\mathbf{h}$: $\langle\text{sha1}\rangle$)
view (recipe + provenance)	ewm-ops program lattice ($\mathbf{p}$: $\langle\text{sha1}\rangle$), vocabulary ($\mathbf{v}$: $\langle\text{sha1}\rangle$); provenance is the list of source $\mathbf{h}$: CIDs
explorer	ewm-sm-explore

Table 6: Concept-to-code correspondence.

- [3] *HLLSet Algebra: A Content-Addressed Set Algebra for Emergent Ontology*, ASTES Journal, 2026.
- [4] G. Birkhoff, *Lattice Theory*, American Mathematical Society Colloquium Publications, 1940.
- [5] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, *HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm*, in *Analysis of Algorithms (AofA)*, 2007.
- [6] B. H. Bloom, *Space/time trade-offs in hash coding with allowable errors*, Communications of the ACM, 1970.
- [7] A. Z. Broder, *On the resemblance and containment of documents*, in *Compression and Complexity of Sequences*, 1997.
- [8] A. Appleby, *MurmurHash3*, 2011; x64-128 variant, lower 64 bits, configurable seed.
- [9] R. C. Merkle, *A digital signature based on a conventional encryption function*, in *Advances in Cryptology (CRYPTO)*, 1988.
- [10] S. Chacon and B. Straub, *Pro Git*, Apress, 2nd ed., 2014.
- [11] J. Benet, *IPFS—Content Addressed, Versioned, P2P File System*, arXiv:1407.3561, 2014.
- [12] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, *Conflict-free replicated data types*, in *Symposium on Self-Stabilizing Systems (SSS)*, 2011.
- [13] D. Ha and J. Schmidhuber, *World Models*, arXiv:1803.10122, 2018.
- [14] Y. LeCun, *A Path Towards Autonomous Machine Intelligence*, OpenReview, 2022.
- [15] A. Vaswani et al., *Attention is all you need*, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [16] P. Lewis et al., *Retrieval-augmented generation for knowledge-intensive NLP tasks*, in *NeurIPS*, 2020.
- [17] W. R. Ashby, *Design for a Brain*, Chapman & Hall, 1952.
- [18] D. Lemire, O. Kaser, N. Kurz, L. Deri, C. O'Hara, F. Saint-Jacques, and G. Ssi-Yan-Kai, *Roaring bitmaps: implementation of an optimized software library*, Software: Practice and Experience, 2018.